
AV1 · ROTEIRO DE VÍDEO

Árvore de Decisão para Campanhas Educativas no Trânsito

Roteiro completo e didático para gravação de vídeo de 5 a 7 minutos, descrevendo o processo de implementação do algoritmo e os resultados do dashboard.

BASE

72.529 sinistros PRF · 2025

ALGORITMO

DecisionTree · criterion=entropy

Roteiro do Vídeo — AV1

Árvore de Decisão para Campanhas Educativas no Trânsito

Como usar este roteiro. A coluna "FALAR" tem o texto literal que deve ser narrado, exatamente como está escrito — pausas e ênfases já estão marcadas. A coluna "MOSTRAR" indica o que deve aparecer na tela em cada momento. Os números entre colchetes **[XX]%** devem ser preenchidos com os valores reais que aparecem no terminal após rodar **python main.py** com o CSV real da PRF.

6 min	7	5-7 min	1080p
Duração alvo	Blocos	Faixa aceita	Resolução

Cronograma geral

Bloco	Tempo	Conteúdo
1	0:00 - 0:30	Contexto e pergunta gerencial
2	0:30 - 1:30	Base de dados + vazamento
3	1:30 - 2:30	Engenharia de atributos
4	2:30 - 3:30	Algoritmo e hiperparâmetros
5	3:30 - 5:00	Resultados: baseline × modelo
6	5:00 - 6:30	Atributos importantes e campanhas
7	6:30 - 7:00	Encerramento

1

Contexto e pergunta gerencial

0:00 – 0:30

MOSTRAR NA TELA:

Tela cheia do dashboard aberto no Hero (KPIs visíveis no topo). Alternativa: slide inicial com o título do trabalho.

"Sinistros em rodovias federais brasileiras geram milhares de vítimas todos os anos. Historicamente, as campanhas educativas de trânsito são definidas pela experiência prática dos gestores. Nosso trabalho substitui essa abordagem intuitiva por um modelo **orientado por dados, auditável e reproduzível**.

A pergunta gerencial que nos guia é: **quais características dos sinistros estão mais associadas à ocorrência de vítimas, e devem ser priorizadas como foco de campanhas educativas?** Para respondê-la, construímos uma Árvore de Decisão treinada com critério de entropia — equivalente ao ID3 clássico."

Dica do roteiro: Fale devagar nesta abertura. A primeira frase define o tom do vídeo inteiro.

2

Base de dados e vazamento

0:30 – 1:30

MOSTRAR NA TELA:

No VS Code, abrir o arquivo **main.py** e rolar até a constante **LEAKAGE_COLS** (linha ~74). Destacar a linha do **assert not set(LEAKAGE_COLS) & set(X.columns)**.

"A base veio da Polícia Rodoviária Federal, via dados.gov.br: **72.529 registros x 30 colunas**, cobrindo os 365 dias de 2025. A leitura exigiu três parâmetros específicos: separador ponto-e-vírgula, encoding latin1 e vírgula decimal — sem eles a acentuação corrompe.

Mas o ponto mais crítico de rigor metodológico está aqui. **As sete colunas pessoas, mortos, feridos_leves, feridos_graves, ilesos, ignorados e feridos são a contagem pós-evento que originou o próprio rótulo.** Incluir qualquer uma delas produziria um modelo com aproximadamente 100% de acurácia artificial — apenas redescobrimos a regra que gerou o rótulo, sem revelar nenhum fator acionável para campanha.

Por isso, no código, há um **assert** explícito que **bloqueia a presença** dessas colunas em X. Se outro profissional tentar adicioná-las no futuro, o pipeline quebra imediatamente, sinalizando o erro. Essa é uma proteção metodológica permanente."

Dica do roteiro: Aponte o cursor do mouse para a linha do assert quando mencioná-la. Mantenha o cursor parado enquanto explica.

3

Engenharia de atributos

1:30 - 2:30

MOSTRAR NA TELA:

No VS Code, mostrar a função **aplicar_engenharia** (linha ~180) e depois a função **binarizar_tracado** (usa `MultiLabelBinarizer`).

"Três transformações de engenharia foram necessárias antes do treinamento.

Primeira: a coluna **tracado_via** tinha **605 valores compostos** — por exemplo, a string **Reta;Declive** em uma única célula. Não é categórica simples, é **multirrótulo**. Usei o **MultiLabelBinarizer** para decompô-la em colunas binárias, uma por palavra-chave: **tracado_Reta**, **tracado_Curva**, **tracado_Aclive** e assim por diante.

Segunda: **horario**, com 1.412 valores no formato HH:MM:SS, virou **hora_int** (0 a 23) e **periodo_dia** em quatro categorias: Madrugada, Manhã, Tarde e Noite. Isso reduz a cardinalidade sem perder a informação sazonal.

Terceira: **data_inversa**, com 365 datas únicas, virou **mes** (1 a 12) para análise sazonal sem explodir a dimensionalidade.

Por fim, **causas raras abaixo de 1%** — aquelas com menos de 725 ocorrências — foram agrupadas em **Outros** antes do one-hot encoding. Isso evita que a árvore aprenda regras para categorias com 2 ou 3 amostras, o que geraria overfitting."

Dica do roteiro: Use os dedos para contar as três transformações. Ajudou quem está assistindo a acompanhar.

4

Algoritmo e hiperparâmetros

2:30 - 3:30

MOSTRAR NA TELA:

Voltar ao dashboard e rolar até a seção "Busca de Hiperparâmetros". Destacar a linha **max_depth=8, min_samples_leaf=30** em azul.

"Usei o **DecisionTreeClassifier** do scikit-learn com **criterion="entropy"** — exatamente o critério de impureza baseado em **ganho de informação** pedido no enunciado, equivalente prático ao ID3 e C4.5 clássicos.

Para os hiperparâmetros, testei um **grid 3 por 3**: **max_depth** em 6, 8 e 10, cruzado com **min_samples_leaf** em 20, 30 e 50. A tabela completa está aqui no dashboard. A escolha padrão foi **max_depth=8** e **min_samples_leaf=30** — melhor equilíbrio entre uma árvore interpretável, que cabe numa página A3 e pode ser mostrada no vídeo, e uma generalização que evita overfitting, com gap entre treino e teste abaixo de 0,10.

E como as classes do alvo são extremamente desbalanceadas — 77,5% feridos, 15,4% sem vítimas e apenas 7,2% fatais — usei **class_weight="balanced"**. Isso repondera as classes inversamente à sua frequência, forçando o modelo a se importar mais com a classe minoritária. E o **random_state=42** garante reprodutibilidade total."

Dica do roteiro: Ao citar "77,5% feridos, 15,4% sem vítimas, 7,2% fatais", segure cada número por 1 segundo. Ajuda a gravar.

5

Resultados: baseline x modelo

3:30 – 5:00

MOSTRAR NA TELA:

Dashboard → rolar até a Matriz de Confusão. Passar o mouse em algumas células para mostrar os tooltips. Depois rolar para o Classification Report.

"Primeiro, o **baseline trivial**: um **DummyClassifier** que sempre prevê a classe majoritária — "Com Vítimas Feridas" — atinge **77,5% de acurácia** sem aprender absolutamente nada. É o piso de comparação. Sem reportar este baseline, qualquer acurácia isolada é enganosa.

Meu modelo treinado chegou a **[XX]% de acurácia global** no conjunto de teste. À primeira vista, parece **menor** que o baseline. Mas esse aparente retrocesso é deliberado e ético: com **class_weight="balanced"**, o modelo sacrifica a acurácia global para **subir o recall da classe minoritária — os fatais**.

Olhem a matriz de confusão. A última linha, classe "Com Vítimas Fatais", mostra que o modelo acerta **[YY]% dos casos fatais**, contra **0% do baseline**. Esse é o ganho real para políticas públicas: identificar sinistros fatais, mesmo errando mais nos sem vítimas.

O classification report confirma: precision de fatais [ZZ], recall [WW] e F1 [KK]. O macro-average cai porque as três classes são tratadas com o mesmo peso — e a majoritária perde um pouco — mas o macro recall sobe para [AA]%, mostrando que o modelo aprende a distinguir as três classes, não apenas a majoritária."

Dica do roteiro: Este é o bloco mais importante. Fale mais devagar e enfatize bem a frase "contra 0% do baseline".

6

Atributos importantes e campanhas

5:00 – 6:30

MOSTRAR NA TELA:

Dashboard → seção "Top 20 Atributos". Passar o mouse em algumas barras para mostrar tooltips. Depois rolar para "Recomendações de Campanha".

"Agora, o coração do trabalho: **quais atributos o modelo treinado considera mais importantes** — não pela frequência bruta, mas pelo **ganho de informação acumulado** em cada split da árvore.

No topo da lista aparecem, em ordem: **[atributo 1]**, com importância [valor]; **[atributo 2]**; **[atributo 3]**; e assim por diante. Cada cor representa a coluna original — laranja para causa do acidente, vermelho para número de veículos, roxo para período do dia, etc.

Para cada um dos 10 atributos mais importantes, gerei automaticamente um bloco de recomendação de campanha. Vejam, por exemplo, o primeiro: o atributo **[nome real]** aparece em **[n]** casos, ou **[X]%** do total. Desses, **[Y]% são fatais** — um **lift de [Z] vezes** comparado à taxa global de 7,2%. Lift maior que 1 significa que o grupo está super-representado em fatais, e deve ser priorizado.

Para cada bloco, há quatro dimensões de campanha: **tipo** (por exemplo, prevenção de direção embriagada), **público-alvo** (condutores jovens 18-34 em saídas noturnas), **momento** (noite e madrugada, fins de semana) e **canal** (blitz educativas, rádio, redes sociais). Essa ligação direta entre atributo do modelo e ação concreta de campanha é o que torna o trabalho acionável para o órgão de trânsito."

Dica do roteiro: Ao citar cada atributo do top 3, pare a fala por 1 segundo. Dá tempo de a câmera focar na barra.

7

Encerramento

6:30 – 7:00

MOstrar NA TELA:

Vista geral do dashboard rolando de cima a baixo. Voltar ao topo (Hero) e parar.

"Em resumo: entregamos um **pipeline reproduzível**, com código versionado, anti-vazamento explícito, baseline de comparação, grid de hiperparâmetros, matriz de confusão interpretável e um **relatório automático de recomendações de campanha** com justificativa numérica.

O dashboard interativo, os três gráficos em PNG, o relatório em markdown e o código completo estão disponíveis no repositório do GitHub, com README documentando todas as decisões metodológicas.

A contribuição principal deste trabalho é mostrar que, mesmo com uma técnica clássica como árvore de decisão, é possível — com rigor metodológico adequado — transformar dados públicos de sinistros em **recomendações concretas e auditáveis** para campanhas educativas de trânsito. Obrigado."

Dica do roteiro: Faça uma pausa de 2 segundos antes de "Obrigado". Encerramento limpo.

Placeholders a preencher antes de gravar

Após rodar **python main.py** com o CSV real, abra **outputs/resultados.json** (ou leia o terminal) e substitua:

Placeholder	Significado	Onde encontrar
[XX]%	Acurácia teste do modelo	KPI "Acurácia — Teste" no dashboard
[YY]%	Recall da classe Fatais	Classification Report, linha "Com Vítimas Fatais", col
[ZZ]	Precision Fatais	Classification Report
[WW]	Recall Fatais (mesmo que YY)	Classification Report
[KK]	F1 Fatais	Classification Report
[AA]%	Macro avg recall	Classification Report, linha "Macro avg"
[atributo 1], [2], [3]	Top 3 features	Seção "Top 20 Atributos" no dashboard
[valor]	Importância do atributo 1	Top 20 Atributos (número ao lado da barra)
[nome real], [n], [X]%, [Y]%, [Z]	Dados do 1º atributo do relatório	relatorio_campanhas.md (bloco 1)

Checklist final de gravação

Antes de gravar

- ☐ CSV real da PRF baixado e **python main.py** rodou com sucesso
- ☐ Placeholders do roteiro preenchidos com números reais
- ☐ Dashboard aberto no navegador com zoom 110%
- ☐ VS Code aberto no **main.py** com zoom 125%
- ☐ Microfone testado (grave 30 seg de teste e ouça)
- ☐ Notificações desligadas (Slack, email, WhatsApp)
- ☐ Ambiente silencioso (sem ar condicionado barulhento, sem gente passando)
- ☐ Roteiro impresso ou em segundo monitor (não ler da mesma tela que grava)

Depois de gravar

- ☐ Edição feita (corte respirações longas, "ééé...", "hum...")
- ☐ Legendas geradas (acessibilidade +)
- ☐ Exportação em 1080p MP4, 30fps, bitrate 5–8 Mbps (arquivo final ~150–300 MB)
- ☐ Upload concluído e link testado em janela anônima
- ☐ Link incluído no email final para **lina@ls4business.com.br**

Hospedagem recomendada: YouTube não listado (faça upload, marque como "Não listado", copie o link). Alternativa: Google Drive (compartilhe com "qualquer pessoa com o link"). Não envie o arquivo MP4 por email — é muito grande.